

AULA 06

Gatilhos, cursores, estruturas de controle e tratamento de exceções

Banco de Dados II — Prof. Alysson

19/09/2026

Antes de começar

Onde estamos na disciplina

O caminho até aqui



O banco **guarda** o dado (01) e **recusa** o dado errado sozinho (02).

Sabemos **perguntar** (03) e sabemos **operar o servidor** (04).

A **regra de negócio** já mora dentro do banco (05).

Falta a regra que dispara sozinha, sem ninguém chamar (06).

Uma frase por aula

Aula	Em uma frase
01	O SGBD guarda o dado e responde por ele.
02	O banco recusa o dado errado, por conta própria.
03	Sabemos perguntar coisas difíceis ao banco.
04	O servidor é um programa, e sabemos onde ele guarda tudo.
05	A regra de negócio pode morar dentro do banco.
06	A regra pode disparar sozinha , sem ninguém chamar.

Aula 01 — o ponto de partida

- **Dado** é o registro bruto; **informação** é o dado com significado
- O **SGBD** existe para guardar, proteger e devolver o dado
- No **modelo relacional**, tudo é tabela: linhas e colunas
- **Chave primária** identifica a linha; **chave estrangeira** liga tabelas
- O **PostgreSQL** é o SGBD da disciplina

Sem esse vocabulário, o resto não se sustenta: gatilho é uma regra que age sobre **linhas** de **tabelas**.

Aula 02 — o banco que se defende

Normalização para evitar repetição; DML para alterar os dados.

E, principalmente, a **integridade declarativa**:

Restrição	O que ela garante
PRIMARY KEY	não há linha repetida nem sem identificação
FOREIGN KEY	não existe item apontando para pedido inexistente
NOT NULL	a coluna não fica vazia
CHECK	o valor obedece a uma condição da própria linha
UNIQUE	não há dois clientes com o mesmo e-mail

Declarativa quer dizer: você **descreve** a regra, e o servidor a cumpre.

Aula 03 — perguntar bem

- **Junção** (JOIN) reúne o que está separado em várias tabelas
- **Subconsulta** usa o resultado de uma consulta dentro de outra
- **Operações de conjunto** (UNION , INTERSECT , EXCEPT) comparam resultados
- **Agregação** (SUM , COUNT , AVG) resume muitas linhas em uma

```
SELECT SUM(quantidade * preco_unitario)
FROM item_pedido
WHERE pedido_id = 1;
```

Este comando vai reaparecer hoje — agora dentro de um gatilho.

Aula 04 — a máquina por dentro

- O PostgreSQL é **cliente-servidor**: o `psql` pede, o servidor responde
- Subimos o servidor em **contêiner** com o Podman, sem instalar nada
- Cada banco tem **esquemas**, e cada esquema guarda tabelas, funções...
- No disco, os dados vivem no **diretório de dados**

```
podman run -d --name bd2-pg -e POSTGRES_PASSWORD=bd2 \  
-e POSTGRES_DB=loja docker.io/library/postgres:17
```

É o mesmo comando do laboratório de hoje.

Aula 05 — a regra dentro do banco

```
CREATE OR REPLACE FUNCTION fn_total_pedido(p_pedido_id integer)
RETURNS numeric
LANGUAGE plpgsql
AS $$
DECLARE
    v_total numeric;
BEGIN
    SELECT COALESCE(SUM(quantidade * preco_unitario), 0)
        INTO v_total
        FROM item_pedido
        WHERE pedido_id = p_pedido_id;
    RETURN v_total;
END;
$$;
```

Cabeçalho, retorno, linguagem, cifrões, DECLARE , BEGIN , END .

Aula 05 — função e procedimento

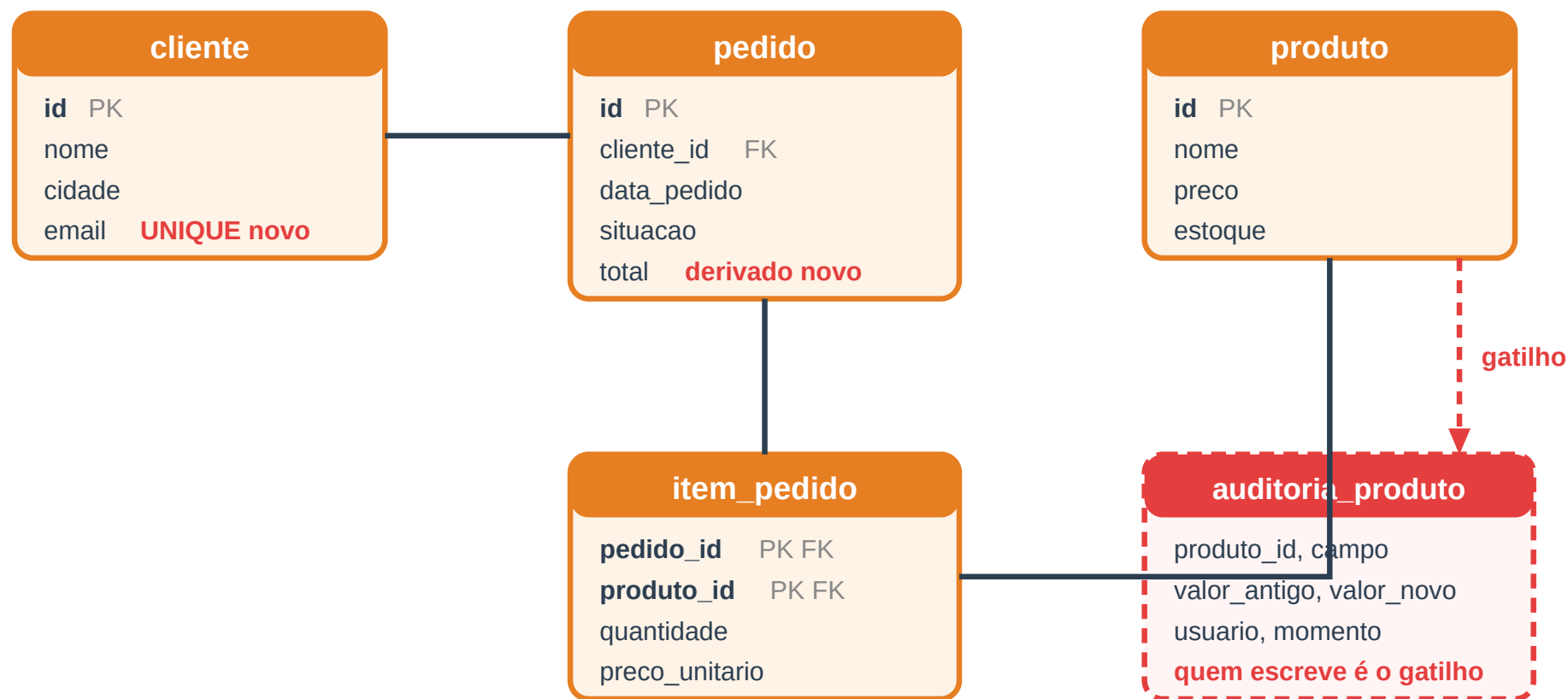
	Função	Procedimento
Devolve valor	sim	não (só por parâmetro)
Como se chama	dentro de um <code>SELECT</code>	com <code>CALL</code>
Transação	não controla	pode confirmar

E as convenções que continuam valendo hoje:

`fn_` função · `sp_` procedimento · `p_` parâmetro · `v_` variável

Hoje entram mais duas: `c_` para cursor e `fn_tg_` para função gatilho.

O esquema da loja



O que mudou no esquema

Continuamos na loja da Aula 05, com **três acréscimos**:

Novidade	Para que serve hoje
<code>cliente.email</code> (<code>UNIQUE</code>)	estudar o erro de valor duplicado
<code>pedido.total</code>	valor derivado , mantido por gatilho
<code>auditoria_produto</code>	trilha de quem mudou o quê e quando

Um esquema cresce junto com as regras do negócio. Isso é normal.

Pergunta de partida

A loja decidiu que **nenhum preço pode subir mais de 50% de uma vez**.

Tentamos escrever a regra como já sabemos:

```
ALTER TABLE produto  
  ADD CONSTRAINT preco_max CHECK (preco <= preco_antigo * 1.5);
```

```
ERROR: column "preco_antigo" does not exist
```

A `CHECK` enxerga **só a linha nova**. Ela não faz ideia de qual era o preço antes.

Quem guarda o valor antigo?

O limite da regra declarativa

A `CHECK`, a `UNIQUE` e a `FOREIGN KEY` resolvem muita coisa — mas **não** estas:

O que precisamos	Por que a restrição não dá conta
comparar o valor antigo com o novo	a restrição só vê a linha nova
registrar quem alterou e quando	restrição não escreve em outra tabela
manter um valor derivado atualizado	restrição não altera dados
corrigir o dado antes de gravar	restrição só aceita ou recusa

Para os quatro casos existe uma só resposta: **gatilho**.

Hoje vamos aprender

1. **Estruturas de controle** — decidir e repetir dentro do PL/pgSQL
2. **Tratamento de exceções** — o que fazer quando dá erro
3. **Cursors** — percorrer o resultado linha a linha
4. **Gatilhos** — a regra que o próprio servidor dispara

Os três primeiros são as **ferramentas**; o quarto é o **objetivo**.

Um gatilho é uma rotina que decide, repete e trata erro — por isso a ordem.

Como esta aula funciona

- **No encontro:** construímos os conceitos e lemos o código juntos
- **Não há atividade em sala:** nada a entregar hoje
- **Em casa:** o laboratório no iximiuz Labs e as dez questões no Classroom

A conta gratuita do iximiuz Labs oferece cerca de **1 hora de ambiente por dia**, e o ambiente é **descartado** ao fim da sessão. Por isso o laboratório está dividido em **dois blocos de ~30 minutos**, e cada bloco **começa do zero** — nada se perde entre um dia e outro.

Estes slides bastam para revisar a aula inteira, inclusive o laboratório.

Parte 1

Estruturas de controle

Por que o SQL sozinho não basta

O SQL é uma linguagem **declarativa**: você descreve o resultado.

```
SELECT nome FROM produto WHERE estoque = 0;
```

Mas algumas regras são **sequências de decisões**:

Se o total passar de R\$ 500, o frete é grátis;
senão, se passar de R\$ 200, o frete é R\$ 15;
senão, o frete é R\$ 30.

O PL/pgSQL acrescenta ao SQL o que falta: **decidir** e **repetir**.

O mapa das estruturas

Estrutura	Para quê
IF / ELSIF / ELSE	escolher um caminho entre vários
CASE	a mesma escolha, escrita como tabela
LOOP ... EXIT	repetir até mandar parar
WHILE ... LOOP	repetir enquanto a condição for verdadeira
FOR i IN 1..10	repetir um número conhecido de vezes
FOR r IN SELECT ...	repetir para cada linha de uma consulta

A última é a ponte para a Parte 3: ela já é um cursor.

IF — a decisão em degraus

```
IF condição1 THEN
    comandos;
ELSIF condição2 THEN
    comandos;
ELSE
    comandos;
END IF;
```

Três detalhes que derrubam quem está começando:

- é **ELSIF**, sem o "E" — não **ELSEIF** nem **ELSE IF**
- o bloco termina com **END IF;**, com espaço e ponto e vírgula
- o primeiro **THEN** verdadeiro vence; os demais nem são testados

IF na prática — a faixa de frete

```
CREATE OR REPLACE FUNCTION fn_faixa_frete(p_total numeric)
RETURNS numeric
LANGUAGE plpgsql
IMMUTABLE
AS $$
DECLARE
    v_frete numeric;
BEGIN
    IF p_total >= 500 THEN
        v_frete := 0;
    ELSIF p_total >= 200 THEN
        v_frete := 15.00;
    ELSE
        v_frete := 30.00;
    END IF;
    RETURN v_frete;
END;
```

Testando a faixa de frete

```
SELECT fn_faixa_frete(600), fn_faixa_frete(300), fn_faixa_frete(50);
```

fn_faixa_frete	fn_faixa_frete	fn_faixa_frete
0	15.00	30.00

Pare e confira: por que `fn_faixa_frete(500)` devolve `0` e não `15.00` ?

Porque `>= 500` é testado **primeiro** e já é verdadeiro. A ordem dos degraus é parte da regra.

CASE — a mesma decisão como tabela

```
RETURN CASE
    WHEN p_estoque = 0 THEN 'esgotado'
    WHEN p_estoque <= 5 THEN 'crítico'
    WHEN p_estoque <= 20 THEN 'normal'
    ELSE 'folgado'
END;
```

- Lê-se de cima para baixo, como o IF
- Cabe **dentro de um SELECT**, o que o IF não faz
- Use quando a decisão for **um valor**; use IF quando forem **vários comandos**

O **CASE** sem **ELSE** é uma armadilha

```
v_nivel := CASE p_estoque
            WHEN 0 THEN 'esgotado'
            WHEN 1 THEN 'último'
            END;
```

Com `p_estoque = 7`, dentro do PL/pgSQL:

```
ERROR: case not found
HINT: CASE statement is missing ELSE part.
```

Regra prática: todo **CASE** termina com **ELSE**, nem que seja **ELSE NULL**.

Os laços — três formas de repetir

```
LOOP                                -- repete sempre; alguém precisa mandar parar
    ...
    EXIT WHEN condição;
END LOOP;

WHILE condição LOOP                -- testa ANTES de cada repetição
    ...
END LOOP;

FOR v_i IN 1..12 LOOP              -- número conhecido de voltas
    ...
END LOOP;
```

No `FOR`, a variável do contador **não é declarada**: ela nasce no próprio `FOR`.

WHILE na prática — quantas parcelas cabem

```
CREATE OR REPLACE FUNCTION fn_parcelas(  
    p_total      numeric,  
    p_valor_minimo numeric DEFAULT 100)  
RETURNS integer  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_n integer := 12;  
BEGIN  
    WHILE v_n > 1 AND p_total / v_n < p_valor_minimo LOOP  
        v_n := v_n - 1;  
    END LOOP;  
    RETURN v_n;  
END;  
$$;
```

Testando as parcelas

```
SELECT fn_parcelas(1200), fn_parcelas(250), fn_parcelas(80);
```

fn_parcelas	fn_parcelas	fn_parcelas
12	2	1

- $1200 / 12 = 100$ → cabe no mínimo, para no teto de 12
- $250 / 2 = 125$ → desceu até caber
- 80 → nunca chega a R\$ 100; a condição $v_n > 1$ impede o laço infinito

Todo laço precisa de uma saída garantida. Aqui são duas.

FOR com contador — e o RAISE NOTICE

```
CREATE OR REPLACE PROCEDURE sp_mostrar_parcelas(  
    p_total numeric, p_vezes integer)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_valor numeric := round(p_total / p_vezes, 2);  
BEGIN  
    FOR v_i IN 1..p_vezes LOOP  
        RAISE NOTICE 'Parcela %/% ..... R$ %', v_i, p_vezes, v_valor;  
    END LOOP;  
END;  
$;
```

RAISE NOTICE escreve na tela **sem interromper** a rotina. É o `print` do banco.

Testando o **FOR**

```
CALL sp_mostrar_parcelas(900, 3);
```

```
NOTICE:   Parcela 1/3 ..... R$ 300.00  
NOTICE:   Parcela 2/3 ..... R$ 300.00  
NOTICE:   Parcela 3/3 ..... R$ 300.00  
CALL
```

Cada % da mensagem é substituído por um valor, **na ordem** em que aparecem.

Três palavras que valem a pena guardar

Palavra	O que faz
EXIT	sai do laço agora
EXIT WHEN cond	sai quando a condição for verdadeira
CONTINUE	pula para a próxima volta, sem terminar esta
FOUND	variável que diz se o último comando achou alguma linha

```
SELECT estoque INTO v_estoque FROM produto WHERE id = p_id;  
IF NOT FOUND THEN  
    RAISE EXCEPTION 'Produto % não encontrado.', p_id;  
END IF;
```

FOUND aparece de novo nos cursores, na Parte 3.

Resumo da Parte 1

- O SQL descreve **o quê**; o PL/pgSQL descreve **como**
- `IF` para vários comandos; `CASE` para escolher **um valor**
- Todo `CASE` merece um `ELSE`
- `WHILE` testa antes; `FOR` conta; `LOOP` precisa de `EXIT`
- `RAISE NOTICE` mostra o que está acontecendo lá dentro
- `FOUND` responde: "o comando anterior achou alguma coisa?"

Parte 2

Tratamento de exceções

O que acontece quando dá erro

Sem tratamento, o erro **aborta o comando inteiro**:

```
SELECT 10 / 0;
```

```
ERROR: division by zero
```

A rotina para no meio. O que já tinha sido alterado **naquele comando** é desfeito, e quem chamou recebe a mensagem técnica do servidor.

Às vezes é exatamente o que queremos. Outras vezes, não.

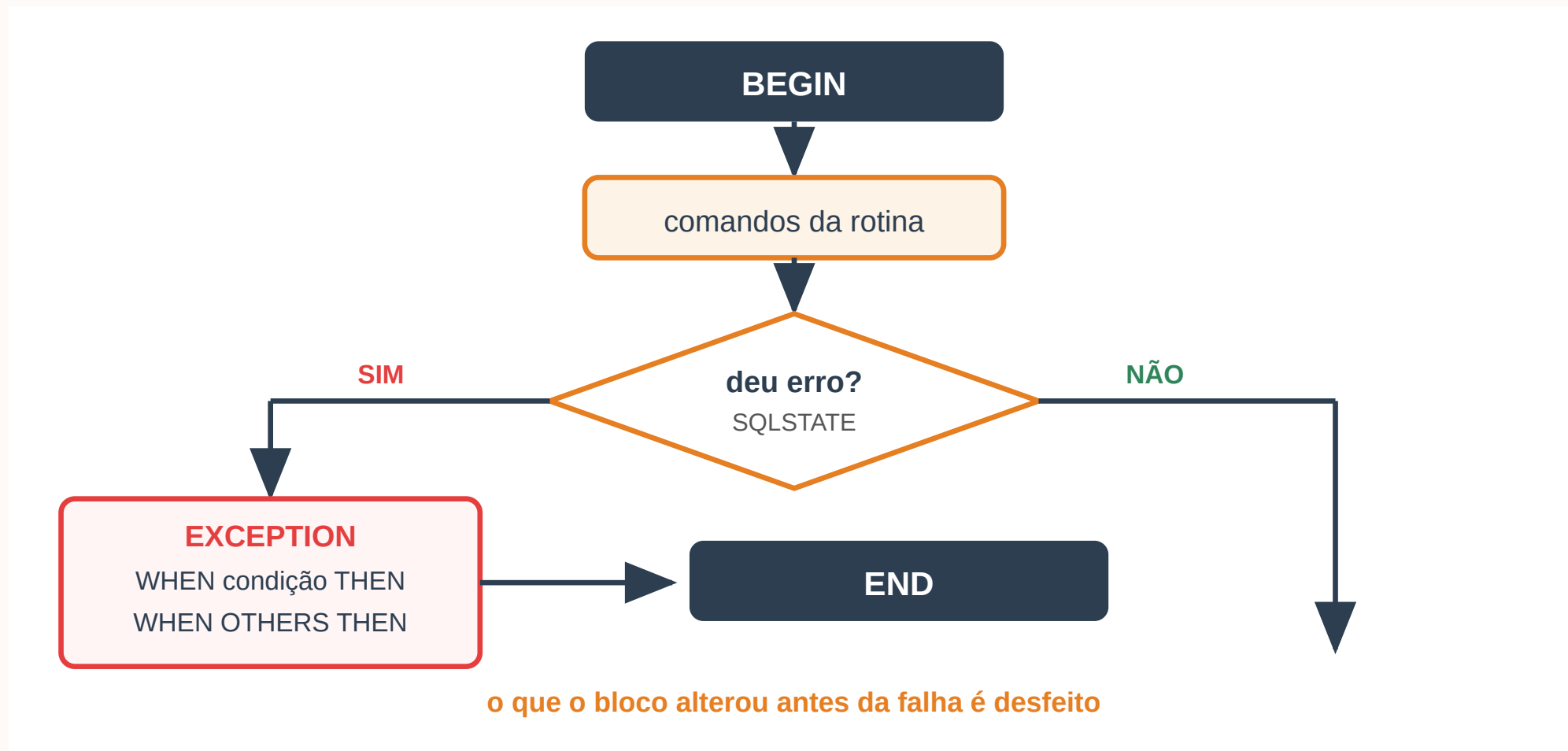
O bloco que segura o erro

```
BEGIN
    -- comandos que podem falhar
EXCEPTION
    WHEN condição THEN
        -- o que fazer quando ESSE erro acontecer
    WHEN OTHERS THEN
        -- qualquer outro erro
END;
```

Três ideias em uma:

1. o erro **não** sobe para quem chamou
2. as alterações feitas **dentro do bloco** são desfeitas
3. a rotina **continua** a partir do tratador

Como o servidor procura o tratador



Exemplo — divisão protegida

```
CREATE OR REPLACE FUNCTION fn_dividir_seguro(  
    p_dividendo numeric, p_divisor numeric)  
RETURNS numeric  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    RETURN round(p_dividendo / p_divisor, 2);  
EXCEPTION  
    WHEN division_by_zero THEN  
        RAISE NOTICE 'Divisor igual a zero; devolvendo NULL.';  
        RETURN NULL;  
END;  
$$;
```

Repare: o `EXCEPTION` fica **dentro** do `BEGIN ... END`, antes do `END`.

Testando a divisão protegida

```
SELECT fn_dividir_seguro(10, 4);  
SELECT fn_dividir_seguro(10, 0);
```

```
fn_dividir_seguro  
-----  
                2.50
```

```
fn_dividir_seguro  
-----
```

```
NOTICE:  Divisor igual a zero; devolvendo NULL.
```

A segunda chamada devolveu `NULL` — a coluna aparece vazia — e a consulta **não** foi abortada.

As condições que mais aparecem

Condição	Quando acontece
<code>division_by_zero</code>	divisão por zero
<code>unique_violation</code>	valor repetido em coluna <code>UNIQUE</code>
<code>foreign_key_violation</code>	referência para linha que não existe
<code>not_null_violation</code>	<code>NULL</code> em coluna <code>NOT NULL</code>
<code>check_violation</code>	valor recusado por uma <code>CHECK</code>
<code>no_data_found</code>	<code>SELECT ... INTO STRICT</code> sem resultado
<code>OTHERS</code>	qualquer erro não listado acima

São os mesmos erros da Aula 02 — agora vistos de dentro da rotina.

Duas variáveis que só existem no tratador

Variável	Conteúdo
SQLSTATE	o código do erro, com cinco caracteres

```
EXCEPTION
  WHEN unique_violation THEN
    RAISE NOTICE 'Código do erro: % – %', SQLSTATE, SQLERRM;
```

```
NOTICE: Código do erro: 23505 – duplicate key value violates
        unique constraint "cliente_email_key"
```

23505 é o código universal de valor duplicado: vale em qualquer SGBD que siga o padrão.

Exemplo — traduzir o erro para o usuário

```
CREATE OR REPLACE PROCEDURE sp_cadastrar_cliente(  
    p_nome text, p_cidade text, p_email text)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    INSERT INTO cliente (nome, cidade, email)  
    VALUES (p_nome, p_cidade, p_email);  
  
    RAISE NOTICE 'Cliente % cadastrado.', p_nome;  
EXCEPTION  
    WHEN unique_violation THEN  
        RAISE NOTICE 'O e-mail % já pertence a outro cliente.', p_email;  
        RAISE NOTICE 'Código do erro: % – %', SQLSTATE, SQLERRM;  
END;  
$$;
```

O atendente lê a primeira mensagem; o suporte técnico lê a segunda.

Testando o cadastro

```
CALL sp_cadastrar_cliente('Diego Alves', 'Gurupi', 'diego@exemplo.com');  
CALL sp_cadastrar_cliente('Outro Diego', 'Palmas', 'diego@exemplo.com');
```

```
NOTICE: Cliente Diego Alves cadastrado.  
CALL  
NOTICE: O e-mail diego@exemplo.com já pertence a outro cliente.  
NOTICE: Código do erro: 23505 – duplicate key value violates ...  
CALL
```

A segunda chamada **não** gravou nada e **não** derrubou a sessão.

Note o `CALL` no fim: para quem chamou, deu tudo certo.

RAISE — falar e interromper

Nível	O que faz
RAISE NOTICE	escreve a mensagem e continua
RAISE WARNING	idem, com destaque maior
RAISE EXCEPTION	escreve a mensagem e interrompe

```
RAISE EXCEPTION 'Estoque insuficiente: há % e foram pedidas %.',  
    v_estoque, p_quantidade  
    USING HINT = 'Reponha o estoque ou reduza a quantidade.';
```

O `USING HINT` acrescenta uma **orientação** do que fazer — quem lê o erro ganha o caminho, não só o problema.

Exemplo — emitir o próprio erro

```
CREATE OR REPLACE PROCEDURE sp_baixar_estoque(  
    p_produto_id integer, p_quantidade integer)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_estoque integer;  
BEGIN  
    SELECT estoque INTO v_estoque FROM produto WHERE id = p_produto_id;  
  
    IF NOT FOUND THEN  
        RAISE EXCEPTION 'Produto % não encontrado.', p_produto_id;  
    END IF;  
  
    IF v_estoque < p_quantidade THEN  
        RAISE EXCEPTION 'Estoque insuficiente: há % unidades e foram pedidas %.',  
            v_estoque, p_quantidade  
            USING HINT = 'Reponha o estoque ou reduza a quantidade.';  
    END IF;  
  
    UPDATE produto SET estoque = estoque - p_quantidade WHERE id = p_produto_id;  
END;
```

Testando a baixa de estoque

```
CALL sp_baixar_estoque(3, 1);      -- monitor: 4 unidades
SELECT id, nome, estoque FROM produto WHERE id = 3;
CALL sp_baixar_estoque(3, 500);
```

id	nome	estoque
3	Monitor	3

ERROR: Estoque insuficiente: há 3 unidades e foram pedidas 500.
HINT: Reponha o estoque ou reduza a quantidade.

A rotina parou **antes** do `UPDATE` : o estoque continua em 3.

Três cuidados com exceções

1. **WHEN OTHERS sozinho esconde defeito.** Capturar tudo e seguir em frente transforma erro de programação em resultado errado silencioso. Se capturar `OTHERS`, ao menos registre o `SQLSTATE`.
2. **O bloco com EXCEPTION custa caro.** O servidor cria um ponto de salvamento a cada entrada no bloco. Dentro de um laço de mil voltas, são mil pontos de salvamento.
3. **Exceção não é desvio de fluxo.** Se dá para testar com `IF` antes, teste com `IF`. Guarde a exceção para o que é, de fato, excepcional.

Resumo da Parte 2

- `BEGIN ... EXCEPTION WHEN ... END` captura o erro e segue
- O que o bloco alterou antes da falha é **desfeito**
- `SQLSTATE` traz o código; `SQLERRM`, a mensagem
- `RAISE NOTICE` informa; `RAISE EXCEPTION` interrompe
- `USING HINT` diz ao usuário **o que fazer**
- Capturar tudo é pior do que não capturar nada

Parte 3

Cursores

Conjunto × linha a linha

EM CONJUNTO

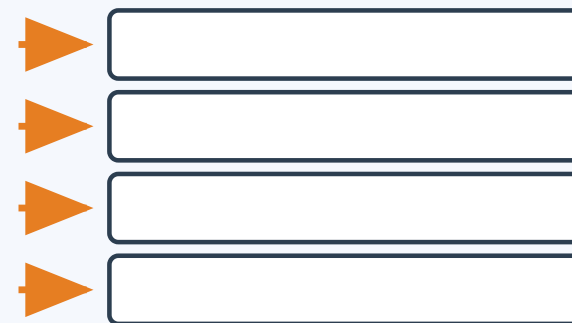
```
UPDATE produto SET preco = preco * 1.1;
```



um comando pega **todas** as linhas
mais curto e mais rápido

COM CURSOR

```
FOR v_linha IN SELECT ... LOOP
```



1, 2, 3, 4...

o marcador visita **uma por vez**
só quando cada linha é diferente

O que é um cursor

Um **cursor** é um marcador que caminha pelo resultado de uma consulta, **uma linha por vez**.

Serve quando cada linha exige um tratamento próprio:

- registrar uma mensagem diferente para cada produto
- chamar outra rotina para cada pedido
- montar um relatório em que cada linha depende da anterior

Quando **não** serve: para alterar todas as linhas do mesmo jeito.

Aí um `UPDATE` resolve — e resolve muito mais rápido.

Cursor implícito — o mais usado

```
FOR v_linha IN
    SELECT id, nome, preco FROM produto ORDER BY id
LOOP
    -- v_linha.id, v_linha.nome, v_linha.preco
END LOOP;
```

O `FOR ... IN SELECT` **abre, percorre e fecha** o cursor sozinho.

- `v_linha` é declarada como `record` e recebe a linha inteira
- os campos são acessados com ponto: `v_linha.nome`
- quando as linhas acabam, o laço termina — sem `EXIT`

Exemplo — reajustar os preços

```
CREATE OR REPLACE PROCEDURE sp_reajustar_precos(p_percentual numeric)
LANGUAGE plpgsql
AS $$
DECLARE
    v_linha record;
    v_novo numeric;
    v_conta integer := 0;
BEGIN
    FOR v_linha IN SELECT id, nome, preco FROM produto ORDER BY id LOOP
        v_novo := round(v_linha.preco * (1 + p_percentual / 100), 2);
        UPDATE produto SET preco = v_novo WHERE id = v_linha.id;
        RAISE NOTICE '% : % -> %', v_linha.nome, v_linha.preco, v_novo;
        v_conta := v_conta + 1;
    END LOOP;

    RAISE NOTICE '% produto(s) reajustado(s).', v_conta;
END;
$;
```

Testando o reajuste

```
CALL sp_reajustar_precos(10);
```

```
NOTICE: Teclado : 120.00 -> 132.00  
NOTICE: Mouse : 80.00 -> 88.00  
NOTICE: Monitor : 900.00 -> 990.00  
NOTICE: Cabo HDMI : 35.00 -> 38.50  
NOTICE: 4 produto(s) reajustado(s).
```

Quatro voltas, quatro `UPDATE`. Guarde esse detalhe: na Parte 4 vamos descobrir quantas vezes um gatilho dispara aqui.

Cursor explícito — as quatro etapas

```
DECLARE
    c_produto CURSOR FOR SELECT id, nome FROM produto;    -- 1. declarar
BEGIN
    OPEN c_produto;                                         -- 2. abrir
    LOOP
        FETCH c_produto INTO v_linha;                     -- 3. buscar
        EXIT WHEN NOT FOUND;
        ...
    END LOOP;
    CLOSE c_produto;                                       -- 4. fechar
END;
```

`FOUND` volta a aparecer: depois do `FETCH`, ela diz se ainda veio linha.

Sem o `EXIT WHEN NOT FOUND`, o laço não termina.

Exemplo — cursor com parâmetro

```
CREATE OR REPLACE PROCEDURE sp_listar_estoque_baixo(p_limite integer DEFAULT 5)
LANGUAGE plpgsql
AS $$
DECLARE
    c_produto CURSOR (p_lim integer) FOR
        SELECT id, nome, estoque FROM produto
        WHERE estoque <= p_lim ORDER BY estoque, id;
    v_linha record;
BEGIN
    OPEN c_produto(p_limite);
    LOOP
        FETCH c_produto INTO v_linha;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Produto % (%) com % unidade(s) – situação: %',
            v_linha.id, v_linha.nome, v_linha.estoque,
            fn_situacao_estoque(v_linha.estoque);
    END LOOP;
    CLOSE c_produto;
END;
```

Testando o cursor explícito

```
CALL sp_listar_estoque_baixo(5);
```

```
NOTICE: Produto 3 (Monitor) com 3 unidade(s) – situação: crítico  
CALL
```

O cursor recebe o limite como **parâmetro**, igual a uma função.

Repare que ele chamou `fn_situacao_estoque`, criada na Parte 1: rotinas armazenadas se combinam.

O mesmo resultado sem cursor

```
CREATE OR REPLACE FUNCTION fn_estoque_baixo(p_limite integer DEFAULT 5)
RETURNS TABLE (produto_id integer, nome text,
                estoque integer, situacao text)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
        SELECT pr.id, pr.nome, pr.estoque, fn_situacao_estoque(pr.estoque)
        FROM produto pr
        WHERE pr.estoque <= p_limite
        ORDER BY pr.estoque, pr.id;
END;
$$;
```

Menos código, e o resultado vem como **tabela**, pronto para outra consulta.

Quando usar cada um

Situação	Escolha
a mesma alteração em todas as linhas	<code>UPDATE</code> direto
devolver linhas para quem chamou	<code>RETURNS TABLE</code> + <code>RETURN QUERY</code>
tratamento diferente por linha	<code>FOR ... IN SELECT</code>
precisa abrir, pausar e fechar com controle	cursor explícito

Regra de bolso: comece pelo SQL de conjunto. Só use cursor quando a lógica de cada linha realmente não couber em um comando.

Resumo da Parte 3

- Cursor percorre o resultado **uma linha por vez**
- `FOR ... IN SELECT` é o cursor implícito — abre e fecha sozinho
- O explícito tem quatro etapas: `DECLARE`, `OPEN`, `FETCH`, `CLOSE`
- `EXIT WHEN NOT FOUND` é o que encerra o laço do cursor explícito
- Cursor pode receber parâmetro
- Se o SQL de conjunto resolve, ele é mais curto e mais rápido

Parte 4

Gatilhos

A regra que ninguém pode contornar

Na Aula 05, a regra ficou dentro do banco — mas **só vale se alguém chamar**:

```
CALL sp_baixar_estoque(3, 1);
```

Se o programador esquecer a chamada, ou se alguém alterar a tabela pelo DBeaver, a regra não acontece.

O **gatilho** resolve isso: ele é disparado pelo **próprio servidor**, sempre que a tabela sofre o evento combinado. Não há como esquecer nem como escapar.

As duas peças de um gatilho

1. A FUNÇÃO GATILHO

```
CREATE FUNCTION fn_tg_exemplo()  
RETURNS trigger  
LANGUAGE plpgsql AS $$  
BEGIN ... RETURN NEW; END;  
$$;
```

sem parâmetros · devolve o tipo **trigger**

2. O CREATE TRIGGER

```
CREATE TRIGGER tg_exemplo  
AFTER UPDATE ON produto  
FOR EACH ROW  
WHEN (condição)  
EXECUTE FUNCTION fn_tg_exemplo();
```

quando · qual evento · qual tabela · por linha

tabela produto

INSERT · UPDATE · DELETE

o servidor dispara sozinho

Peça 1 — a função gatilho

```
CREATE OR REPLACE FUNCTION fn_tg_auditar_produto()  
RETURNS trigger          -- tipo especial, obrigatório  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    ...  
    RETURN NEW;  
END;  
$$;
```

Duas diferenças em relação às funções da Aula 05:

- **não recebe parâmetros** entre os parênteses
- **devolve** `trigger`, um tipo que só existe para isso

Peça 2 — o comando **CREATE TRIGGER**

```
CREATE TRIGGER tg_produto_auditar      -- nome do gatilho
  AFTER INSERT OR UPDATE OR DELETE    -- quando e em qual evento
  ON produto                          -- em qual tabela
  FOR EACH ROW                        -- por linha ou por comando
  EXECUTE FUNCTION fn_tg_auditar_produto();
```

Leia como uma frase:

*depois de inserir, alterar ou excluir na tabela produto,
para cada linha, execute esta função.*

As variáveis que o gatilho ganha de graça

Variável	Conteúdo
NEW	a linha como vai ficar (INSERT e UPDATE)
OLD	a linha como estava (UPDATE e DELETE)
TG_OP	'INSERT' , 'UPDATE' , 'DELETE' OU 'TRUNCATE'
TG_TABLE_NAME	o nome da tabela que disparou
TG_WHEN	'BEFORE' , 'AFTER' OU 'INSTEAD OF'
TG_LEVEL	'ROW' OU 'STATEMENT'

Ninguém declara nem preenche essas variáveis: o servidor entrega prontas.

OLD e NEW não existem sempre

operação	OLD	NEW
INSERT	 não existe	a linha que está entrando
UPDATE	a linha como estava	a linha como vai ficar
DELETE	a linha que está saindo	 não existe

O erro mais comum da matéria

Operação	OLD	NEW
INSERT	não existe	a linha que está entrando
UPDATE	a linha antes	a linha depois
DELETE	a linha que sai	não existe

Ler `NEW` num `DELETE` **não dá erro**: vale `NULL`, e o gatilho segue **sem avisar**.
(Só até o PostgreSQL 10 dava `record "new" is not assigned yet`.)

Por isso toda função gatilho que atende mais de um evento **começa testando**
`TG_OP`.

BEFORE × AFTER

	BEFORE	AFTER
Quando dispara	antes de gravar	depois de gravar
Pode alterar NEW	sim	não adianta
Pode cancelar a linha	sim (RETURN NULL)	não
Enxerga o dado já gravado	não	sim
Serve para	corrigir e validar	registrar e propagar

Regra prática: mexer no dado → BEFORE . Reagir ao dado → AFTER .

FOR EACH ROW × FOR EACH STATEMENT

```
UPDATE produto SET preco = preco * 1.1; -- 4 produtos na tabela
```

Tipo	Quantas vezes dispara
FOR EACH ROW	4 vezes — uma por linha
FOR EACH STATEMENT	1 vez — o comando é um só

- No gatilho de **comando** não existem **OLD** nem **NEW**
- **FOR EACH STATEMENT** é o padrão quando você não escreve nada
- Um **UPDATE** que não altera nenhuma linha **ainda dispara** o de comando

A regra do retorno

Tipo de gatilho	O que devolver
<code>BEFORE ... ROW</code>	<code>NEW</code> para seguir; <code>NULL</code> para cancelar a linha
<code>BEFORE DELETE ... ROW</code>	<code>OLD</code> para seguir; <code>NULL</code> para cancelar
<code>AFTER ... ROW</code>	o retorno é ignorado — use <code>RETURN NULL</code>
<code>... STATEMENT</code>	o retorno é ignorado — use <code>RETURN NULL</code>

O cancelamento é **silencioso**: o comando responde `INSERT 0 0` e ninguém recebe erro. Use com cuidado — e documente.

Exemplo 1 — corrigir o dado antes de gravar

```
CREATE OR REPLACE FUNCTION fn_tg_normalizar_cliente()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.nome := btrim(NEW.nome);           -- tira espaços das pontas
    NEW.cidade := btrim(NEW.cidade);
    NEW.email := lower(btrim(NEW.email)); -- e-mail sempre minúsculo

    IF NEW.nome = '' THEN
        RAISE EXCEPTION 'O nome do cliente não pode ser vazio.'
            USING HINT = 'Informe o nome completo do cliente.';
    END IF;

    RETURN NEW; -- devolve a linha JÁ corrigida
END;
$$;

CREATE TRIGGER tg_cliente_normalizar
BEFORE INSERT OR UPDATE ON cliente
```

Testando a normalização

```
INSERT INTO cliente (nome, cidade, email)
VALUES (' Elena Rocha ', 'Palmas', ' ELENA@Exemplo.COM ');

SELECT id, nome, email FROM cliente ORDER BY id DESC LIMIT 1;
```

id	nome	email
6	Elena Rocha	elena@exemplo.com

Ninguém pediu para limpar o dado. O gatilho fez isso **para qualquer programa** que grave nessa tabela.

Testando a validação

```
INSERT INTO cliente (nome, cidade, email)
VALUES ('', 'Palmas', 'vazio@exemplo.com');
```

```
ERROR:  O nome do cliente não pode ser vazio.
HINT:  Informe o nome completo do cliente.
CONTEXT:  PL/pgSQL function fn_tg_normalizar_cliente() line 8 at RAISE
```

A linha `CONTEXT` diz **qual gatilho** recusou e **em que linha do código**.
É por ela que se descobre uma regra escondida.

Exemplo 2 — a regra que a **CHECK** não alcança

Voltamos à pergunta de partida: preço não sobe mais de 50% de uma vez.

```
CREATE OR REPLACE FUNCTION fn_tg_limitar_reajuste()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_variacao numeric;
BEGIN
    v_variacao := round((NEW.preco - OLD.preco) / OLD.preco * 100, 2);

    IF v_variacao > 50 THEN
        RAISE EXCEPTION
            'Reajuste de % por cento no produto % excede o limite de 50 por cento.',
            v_variacao, OLD.nome
            USING HINT = 'Reajustes maiores exigem autorização da diretoria.';
    END IF;

    RETURN NEW;
END;
```

A cláusula **WHEN** — filtrar antes de chamar

```
CREATE TRIGGER tg_produto_limitar_reajuste
  BEFORE UPDATE ON produto
  FOR EACH ROW
  WHEN (NEW.preco IS DISTINCT FROM OLD.preco)
  EXECUTE FUNCTION fn_tg_limitar_reajuste();
```

- A função **só é chamada** se o preço realmente mudou
- Um **UPDATE** que mexe apenas no estoque nem incomoda o gatilho
- **IS DISTINCT FROM** compara certo mesmo quando há **NULL**

Economia de verdade: é a diferença entre chamar a função mil vezes ou nenhuma.

Testando o limite de reajuste

```
CALL sp_reajustar_precos(10);           -- 10% – passa
UPDATE produto SET preco = preco * 2 WHERE id = 1;  -- 100% – não passa
```

```
NOTICE:  Teclado : 120.00 -> 132.00
```

```
...
```

```
ERROR:  Reajuste de 100.00 por cento no produto Teclado excede  
        o limite de 50 por cento.
```

```
HINT:   Reajustes maiores exigem autorização da diretoria.
```

O procedimento da Parte 3 e o gatilho convivem: cada `UPDATE` do laço passa pela conferência.

Exemplo 3 — auditoria com TG_OP

```
CREATE OR REPLACE FUNCTION fn_tg_auditar_produto()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO auditoria_produto (produto_id, operacao, campo, valor_novo)
        VALUES (NEW.id, TG_OP, 'registro', NEW.nome);

    ELSIF TG_OP = 'UPDATE' THEN
        IF NEW.preco IS DISTINCT FROM OLD.preco THEN
            INSERT INTO auditoria_produto
            (produto_id, operacao, campo, valor_antigo, valor_novo)
            VALUES (NEW.id, TG_OP, 'preco', OLD.preco::text, NEW.preco::text);
        END IF;
        -- o mesmo para estoque
    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO auditoria_produto (produto_id, operacao, campo, valor_antigo)
        VALUES (OLD.id, TG_OP, 'registro', OLD.nome);
    END IF;

    RETURN NULL; -- gatilho AFTER: o retorno é ignorado
END;
```

Testando a trilha de auditoria

```
SELECT id, produto_id, operacao, campo, valor_antigo, valor_novo
FROM auditoria_produto ORDER BY id;
```

id	produto_id	operacao	campo	valor_antigo	valor_novo
1	3	UPDATE	estoque	4	3
2	1	UPDATE	preco	120.00	132.00
3	2	UPDATE	preco	80.00	88.00
4	3	UPDATE	preco	900.00	990.00
5	4	UPDATE	preco	35.00	38.50

A linha 1 veio do `CALL sp_baixar_estoque(3, 1)`. As demais, do reajuste.

Ninguém escreveu nessa tabela: o gatilho escreveu.

Exemplo 4 — gatilho de comando

```
CREATE OR REPLACE FUNCTION fn_tg_resumo_produto()  
RETURNS trigger LANGUAGE plpgsql AS $$  
DECLARE  
    v_quantos integer;  
BEGIN  
    SELECT count(*) INTO v_quantos FROM produto;  
    RAISE NOTICE 'Comando % concluído em %; a tabela tem % produto(s).',  
        TG_OP, TG_TABLE_NAME, v_quantos;  
    RETURN NULL;  
END; $$  
  
CREATE TRIGGER tg_produto_resumo  
AFTER UPDATE ON produto  
FOR EACH STATEMENT  
EXECUTE FUNCTION fn_tg_resumo_produto();
```

Quantas vezes ele dispara no reajuste?

`sp_reajustar_precos` executa **um UPDATE** **por volta do laço** — quatro comandos, não um só. Então:

```
NOTICE: Comando UPDATE concluído em produto; a tabela tem 4 produto(s).  
NOTICE: Teclado : 120.00 -> 132.00  
NOTICE: Comando UPDATE concluído em produto; a tabela tem 4 produto(s).  
NOTICE: Mouse : 80.00 -> 88.00  
...
```

Quatro vezes. "Uma vez por comando" não é o mesmo que "uma vez por rotina". E repare na ordem: o gatilho `AFTER` do `UPDATE` termina **antes** do `RAISE NOTICE` que vem depois dele no laço.

Exemplo 5 — manter um valor derivado

```
CREATE OR REPLACE FUNCTION fn_tg_total_pedido()  
RETURNS trigger LANGUAGE plpgsql AS $$  
DECLARE  
    v_pedido integer;  
BEGIN  
    IF TG_OP = 'DELETE' THEN  
        v_pedido := OLD.pedido_id;      -- em DELETE só existe OLD  
    ELSE  
        v_pedido := NEW.pedido_id;  
    END IF;  
  
    UPDATE pedido SET total = fn_total_pedido(v_pedido) WHERE id = v_pedido;  
  
    RETURN NULL;  
END; $$  
  
CREATE TRIGGER tg_item_total  
AFTER INSERT OR UPDATE OR DELETE ON item_pedido  
FOR EACH ROW EXECUTE FUNCTION fn_tg_total_pedido();
```

Testando o valor derivado

```
SELECT id, total FROM pedido WHERE id = 1;
INSERT INTO item_pedido VALUES (1, 2, 1, 80.00);
SELECT id, total FROM pedido WHERE id = 1;
DELETE FROM item_pedido WHERE pedido_id = 1 AND produto_id = 2;
SELECT id, total FROM pedido WHERE id = 1;
```

total	total	total
-----	-----	-----
275.00	355.00	275.00

Inseriu, subiu. Apagou, voltou. **Sem nenhuma linha de programa fazendo a conta.**

O gatilho vale do presente para a frente

Quando criamos `tg_item_total`, os pedidos antigos continuavam com `total = 0`: o gatilho não existia quando eles foram gravados.

```
UPDATE pedido p SET total = fn_total_pedido(p.id);
```

Este comando roda **uma única vez**, para acertar o passado.

Ponto-chave: criar o gatilho não conserta o histórico. Todo projeto que passa a manter valor derivado precisa desse acerto inicial.

Ordem de disparo: alfabética

Quando dois gatilhos atendem ao mesmo evento na mesma tabela, o PostgreSQL os dispara **em ordem alfabética de nome**.

```
CREATE TRIGGER tg_zz_segundo BEFORE UPDATE ON produto ...  
CREATE TRIGGER tg_aa_primeiro BEFORE UPDATE ON produto ...
```

```
NOTICE: sou o gatilho A  
NOTICE: sou o gatilho B
```

Não é a ordem de criação. Se a ordem importar para a regra, **ela precisa estar no nome** — e isso é um sinal de que talvez deveriam ser um só.

Inspeccionar o que existe

```
\d produto
```

Triggers:

```
tg_produto_auditar AFTER INSERT OR DELETE OR UPDATE ON produto
    FOR EACH ROW EXECUTE FUNCTION fn_tg_auditar_produto()
tg_produto_limitar_reajuste BEFORE UPDATE ON produto FOR EACH ROW
    WHEN (new.preco IS DISTINCT FROM old.preco) EXECUTE FUNCTION ...
tg_produto_resumo AFTER UPDATE ON produto FOR EACH STATEMENT ...
```

Fora do `psql`, a mesma informação vem do catálogo:

```
SELECT trigger_name, event_manipulation, action_timing, action_orientation
FROM information_schema.triggers WHERE event_object_table = 'produto';
```

Desligar e remover

```
ALTER TABLE produto DISABLE TRIGGER tg_produto_limitar_reajuste;  
-- carga de dados, correção em massa, importação...  
ALTER TABLE produto ENABLE TRIGGER tg_produto_limitar_reajuste;  
  
DROP TRIGGER tg_produto_resumo ON produto;
```

- No `DROP TRIGGER`, informe **a tabela** — o nome sozinho não basta
- Apagar a tabela apaga os gatilhos dela; a **função** continua existindo
- Desligar gatilho é decisão de manutenção: anote e **religue**

Onde o gatilho ajuda

- **Auditoria:** registrar quem alterou o quê, sem confiar na aplicação
- **Normalização:** padronizar o dado na entrada
- **Regra entre valores:** comparar o antigo com o novo
- **Valor derivado:** manter um total, um saldo, um contador
- **Integridade que a `FOREIGN KEY` não expressa:** regras entre tabelas com condição

O ponto comum: são regras que **não podem depender de quem está chamando.**

Onde o gatilho atrapalha

- **Lógica escondida:** o `UPDATE` faz coisas que o código não mostra
- **Custo por linha:** um gatilho de linha num `UPDATE` de milhões custa caro
- **Cascata:** gatilho que altera tabela com gatilho que altera outra...
- **Depuração difícil:** o erro aparece longe de onde foi causado
- **Ordem alfabética:** dependência entre gatilhos é frágil

Documente todo gatilho. Quem for manter o sistema daqui a dois anos não vai adivinhar que ele existe.

Quem resolve o quê

Precisa de...	Use
valor obrigatório, único ou dentro de uma faixa	NOT NULL , UNIQUE , CHECK
referência entre tabelas	FOREIGN KEY
comparar antigo com novo, auditar, derivar	gatilho
cálculo chamado explicitamente	função ou procedimento
regra de tela, fluxo, mensagem ao usuário	a aplicação

Suba um degrau por vez. Só use gatilho quando a restrição declarativa não alcançar — ela é mais simples, mais rápida e mais visível.

Uma palavra sobre **INSTEAD OF**

Existe um terceiro momento, além de **BEFORE** e **AFTER** :

```
CREATE TRIGGER tg_vw_cliente INSTEAD OF INSERT ON vw_cliente  
FOR EACH ROW EXECUTE FUNCTION fn_tg_gravar_cliente();
```

- Só funciona em **visões**, nunca em tabelas
- Serve para tornar gravável uma visão que junta várias tabelas
- Sempre **FOR EACH ROW**

Guarde o nome: as **visões** são assunto da Aula 11.

Resumo da Parte 4

- Gatilho = **função gatilho** (RETURNS trigger) + **CREATE TRIGGER**
- Quem dispara é o servidor; ninguém precisa lembrar de chamar
- NEW , OLD e TG_OP chegam prontos — e OLD / NEW nem sempre existem
- BEFORE corrige e cancela; AFTER registra e propaga
- ROW roda por linha; STATEMENT , por comando
- WHEN evita chamadas desnecessárias
- Gatilho é poderoso e **invisível**: por isso, documente

Parte 5

O estudo em casa

O combinado

Hoje, em sala: nada a entregar. Vimos os conceitos e lemos o código.

Em casa, até a próxima aula:

1. Rerler estes slides — eles têm o laboratório inteiro
2. Ler o texto-base, em blocos curtos
3. Fazer o **laboratório** no iximiuz Labs
4. Responder às **dez questões** no Google Classroom

Tudo o que você precisa executar está descrito passo a passo, com o resultado esperado de cada comando.

O ambiente da Aula 06

1. Acesse <https://labs.iximiuz.com/playgrounds/BD2-AULA06-c705669c>
2. Clique em **Start Playground** e aguarde a máquina subir
3. Use a aba **Term** para digitar; **Ctrl + Shift + V** para colar

O PostgreSQL 17 já vem baixado e os três scripts já estão em `~/lab06` .
Nada é instalado no seu computador.

Matricule-se uma vez na turma:

<https://labs.iximiuz.com/trainings/aulas-bd2-2026-2-51d1c714>

Como aproveitar a hora que você tem

A conta gratuita dá cerca de **1 hora por dia**, e ao sair o ambiente é **destruído**: o banco e tudo o que você digitou desaparecem.

Antes de iniciar a máquina	Enquanto ela roda	Ao sair
leia o roteiro inteiro	digite, não fique lendo	leve as anotações
deixe os comandos à mão	anote cada resultado	o resto some

Isso não atrapalha, porque **cada bloco começa do zero**: os três scripts voltam para `~/lab06` toda vez que a máquina sobe, e o Bloco B reconstrói o banco inteiro em três comandos.

Não tenha receio de fechar. Reabrir custa um minuto.

Bloco A — controle, exceções e cursores

```
podman run -d --name bd2-pg -e POSTGRES_PASSWORD=bd2 \
  -e POSTGRES_DB=loja docker.io/library/postgres:17
podman exec -i bd2-pg psql -U postgres -d loja < ~/lab06/01_esquema.sql
podman exec -it bd2-pg psql -U postgres -d loja
```

Depois, dentro do `psql` :

1. escreva `fn_faixa_frete` e teste com `600` , `300` e `50`
2. escreva `fn_situacao_estoque` e `fn_parcelas`
3. escreva `fn_dividir_seguro` e divida por zero **de propósito**
4. escreva `sp_baixar_estoque` e peça 500 unidades **de propósito**
5. escreva `sp_reajustar_precos` e rode com `10`
6. escreva `sp_listar_estoque_baixo` com cursor explícito

Confira cada uma em `~/lab06/02_rotinas.sql` **depois** de escrevê-la.

Bloco B — gatilhos

Comece **do zero**, mesmo que tenha acabado o Bloco A agora:

```
podman exec -i bd2-pg psql -U postgres -d loja < ~/lab06/01_esquema.sql
podman exec -i bd2-pg psql -U postgres -d loja < ~/lab06/02_rotinas.sql
podman exec -i bd2-pg psql -U postgres -d loja < ~/lab06/03_gatilhos.sql
podman exec -it bd2-pg psql -U postgres -d loja
```

É o que faz os resultados baterem com o roteiro. Depois, no `psql`:

1. `\d produto` — veja a seção **Triggers**
2. insira um cliente com espaços e maiúsculas: confira a normalização
3. tente um `UPDATE` que dobre o preço: leia o erro **e o CONTEXT**
4. rode `CALL sp_reajustar_precos(10);` e conte os avisos do gatilho de comando
5. leia a tabela `auditoria_produto`
6. insira e apague um item de pedido: acompanhe `pedido.total`

O que anotar

Você vai perder o ambiente; não perca o aprendizado.

- o **comando** que você digitou
- o **resultado** que apareceu
- a **mensagem de erro** completa, quando houver — inclusive `HINT` e `CONTEXT`
- a **dúvida** que ficou

Um caderno, o bloco de notas do celular ou uma foto da tela: qualquer um serve. Isso é o que você vai reler para estudar — e é o que a A1 cobra.

Se você só tem o celular

O playground abre no navegador do celular, mas digitar é difícil.

Uma alternativa honesta:

- **leia** cada passo destes slides e do texto-base
- **copie à mão** o código em um caderno — copiar código ensina
- **responda** por escrito: o que este comando faz? qual o resultado esperado?
- confira a sua resposta no slide "Testando..." correspondente

Quem estudar assim consegue responder às dez questões. O laboratório aprofunda, mas os slides são autossuficientes.

Checklist do laboratório

- [] Subi o PostgreSQL e carreguei o esquema da loja
- [] Usei `IF`, `CASE`, `WHILE` e `FOR` em rotinas minhas
- [] Capturei um erro com `EXCEPTION` e li o `SQLSTATE`
- [] Emiti meu próprio erro com `RAISE EXCEPTION ... USING HINT`
- [] Percorri linhas com cursor implícito e com cursor explícito
- [] Criei um gatilho `BEFORE` que corrige o dado
- [] Criei um gatilho `AFTER` que grava auditoria
- [] Vi `pedido.total` mudar sozinho
- [] Listei, desabilitei e removi um gatilho

Erros mais comuns (1/2)

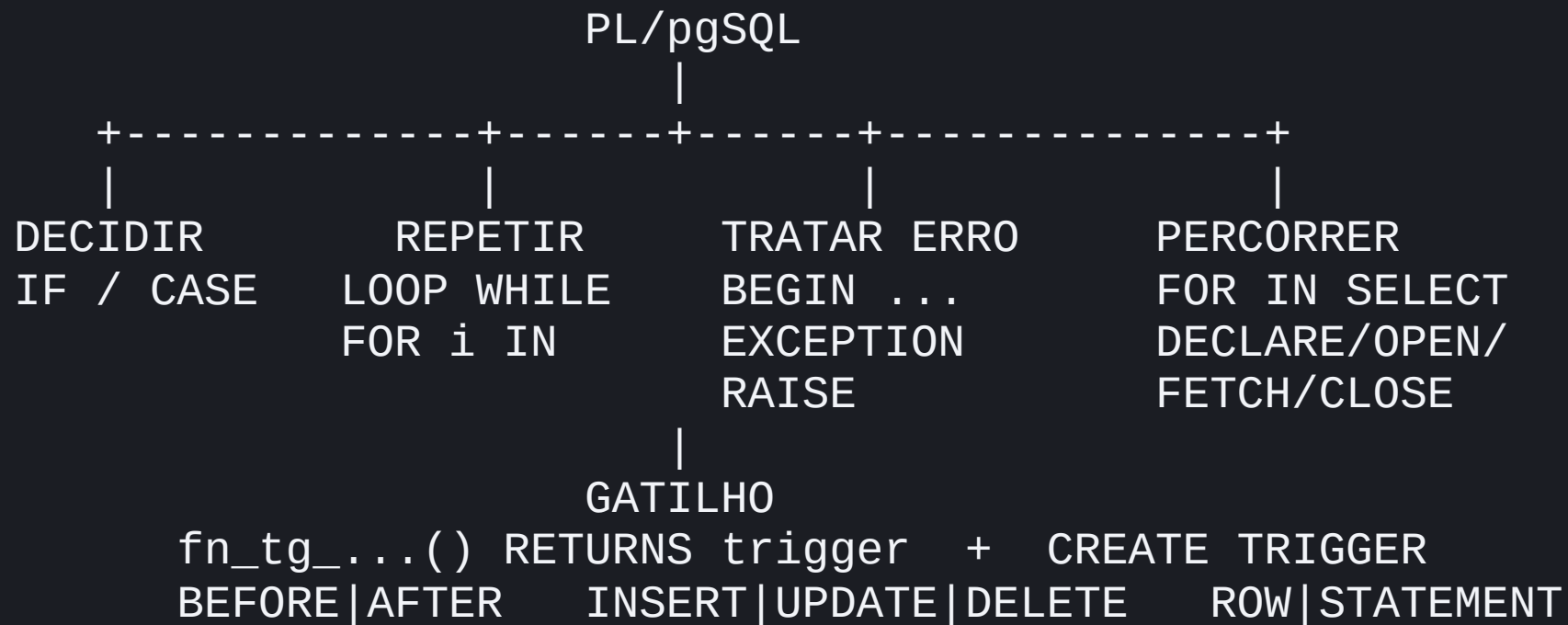
Mensagem	Causa provável
case not found	CASE sem ELSE
record "new" is not assigned yet	leu NEW em DELETE (só até a versão 10; hoje NEW vale NULL , sem erro)
record "old" is not assigned yet	leu OLD em INSERT (só até a versão 10; hoje OLD vale NULL , sem erro)
function ... must return type trigger	esqueceu RETURNS trigger
trigger functions cannot have declared arguments	pôs parâmetro na função gatilho

Erros mais comuns (2/2)

- Escrever `ELSEIF` ou `ELSE IF` em vez de `ELSIF`
- Esquecer `END IF;`, `END LOOP;` ou `END;`
- Montar cursor explícito sem `EXIT WHEN NOT FOUND` → laço infinito
- Fazer `DROP TRIGGER` sem dizer `ON tabela`
- Esperar que o gatilho conserte os dados **antigos** da tabela
- Criar um gatilho `AFTER` e tentar alterar `NEW` dentro dele

Dica: leia a mensagem inteira. O `CONTEXT` diz qual rotina e qual linha.

Mapa-resumo



Para guardar

- `IF` / `CASE` decidem; `LOOP` / `WHILE` / `FOR` repetem
- `EXCEPTION` captura o erro; `RAISE` fala e, se preciso, interrompe
- Cursor percorre linha a linha — mas o SQL de conjunto vem primeiro
- Gatilho são **duas peças**: a função `RETURNS trigger` e o `CREATE TRIGGER`
- `BEFORE` corrige e cancela; `AFTER` registra e propaga
- `NEW` não existe em `DELETE`; `OLD` não existe em `INSERT`
- O gatilho vale **do presente para a frente**
- Ele é invisível para quem chama: **documento sempre**

Atividade de consolidação

- Refaça o laboratório, nos dois blocos
- Escreva **um gatilho seu** sobre o esquema da loja
(sugestão: impedir que um pedido já `'fechado'` receba item novo)
- Responda às **dez questões** no Google Classroom
- Turma no Classroom: <https://classroom.google.com/c/ODY5MzE1NDkyODUx>
- Turma no iximiuz Labs:
<https://labs.iximiuz.com/trainings/aulas-bd2-2026-2-51d1c714>

Traga suas dúvidas para o próximo encontro — começamos por elas.

Encerramento

**Na próxima aula: transações,
propriedades ACID e o protocolo de duas
fases**

Dúvidas?